Chapter 15
Cloud Computing and the Internet of Things

THE FOLLOWING CEH EXAM TOPICS ARE COVERED IN THIS CHAPTER:

- ✓ **Cloud computing threats and attacks**
- ✓ **Cloud computing**
- ✓ **Cloud deployment models**
- ✓ **Internet of Things (IoT)**

It is becoming harder to find businesses today that are not using some form of cloud service. This may be as simple as using a cloud provider for email, or it could be they have multiple web applications they have deployed into a cloud environment using cloud-native application designs. Because the infrastructure is moving to cloud providers, attacks are also moving to these environments. Cloud service providers are also enabling a lot of capabilities for application development that wouldn't typically be available for businesses because the investment levels for the required infrastructure would be too high. Beyond the infrastructure costs, there is also the skill level needed to be able to implement those capabilities. They don't come for nothing, after all.

Cloud computing offers a quick and easy way for companies to outsource their information technology needs. You can stand up a lot of infrastructure quickly without overhead costs. This opens doors for small companies to get started quickly and inexpensively with the same sort of systems and services larger companies get access to. This is what is driving so many companies to cloud services and why it's essential that you understand the different types of cloud services available and the ways they are used.

Cloud providers are also introducing capabilities to support consumer-oriented, network-enabled, embedded devices. These devices, which are used all over today, are commonly called the Internet of Things collectively. Both consumers and businesses may have a lot of instances of

these devices on their networks. These devices are being used to create large botnets if the devices can be compromised. Often, these devices are communicating with cloud providers, which may provide an avenue to attack the devices themselves. Understanding the devices, how they are communicating, and the underlying operating systems is important from the perspective of an ethical hacker.

Cloud Computing Overview

In essence, cloud computing is just another form of outsourcing, which has been happening since at least the 1960s. Back then, you outsourced to a service bureau that could afford the millions of dollars it took to purchase a mainframe computer. This was a large computer system that took an entire room and came in several refrigerator units. Figure 15.1 shows a portion of an IBM 360, which was a popular mainframe computer developed in the 1960s and 1970s.

FIGURE 15.1 IBM System/360 mainframe

Source: Erik Pitti / Wikimedia Commons / CC BY 2.0.

You start with companies that have had to buy more computing power than they need perhaps and decide to sell it. Just like companies who started up service bureau divisions decades ago, companies that had a need for a lot of computing resources that weren't in demand all the time could similarly sell access to those resources to others. One enormous difference, and this probably can't be overstated, is the cost, which has allowed individuals to make use of these services where service bureaus would typically only sell services to businesses simply because individuals didn't have the need for the types of computing services being offered. On top of that, they were expensive.

We have the computing part sorted out. We have lots of computers, much lower cost than those from the 1950s and 1960s, and the space to put all of those computers. This leads us to another difference between service bureaus and what we are talking about here. There were no networks in the early 1960s. Even later, when networks were starting to be construc-

ted in the late 1960s and into the 1970s, businesses didn't have network connections to service bureaus. At best, they may have used phone lines and modems to transmit or receive data. More likely, they were sending magnetic tapes back and forth between the company and the service bureau.

At the time, you could probably get a service bureau to do any computing task you could conceive of. This was probably limited in conception because people were still trying to figure out just what to use these things for. In the early '80s, I worked for a service bureau that primarily managed and printed address labels from mailing lists. The address labels would then be used to apply to catalogs from large mail-order outfits. At the time, personal computers were just getting started, and it was simply easier to pay a service bureau to keep track of nine-track magnetic tapes than to buy the computer hardware and storage capacity needed to store potentially millions of names and addresses. Figure 15.2 shows a small nine-track magnetic tape. Imagine dozens of these at a time where the largest disk drive available for a personal computer may have been 5 MB.

FIGURE 15.2 Nine-track magnetic tape

We'll get to describing those services shortly. First, what is it that qualifies something as a cloud service? Cloud computing services typically have a number of characteristics in common, regardless of the type of service being used. First, cloud computing services are typically accessed and managed using web-based technologies. You would manage or access the service through a web-based interface using standard protocols like the HyperText Transfer Protocol (HTTP) and represented with a presentation language like the Hypertext Markup Language (HTML). This generally allows ubiquitous access from anywhere in the world because you would get access to the service using your web browser over the Internet.

Cloud computing services also typically allow rapid and agile delivery, meaning if you need a service, like a server, for instance, you could go to a provider and request one to be created for you, and it would be available almost instantly—certainly, faster than hours, days, or weeks as might be the case if you needed to go through a provisioning process for yourself, either personally or as a business. As you will see later, you can also get granular access, meaning you only get the service you specifically want. Not everyone wants an entire server to manage and install whatever they want. Sometimes, you want just some storage space or access to an application. Everything else is just a lot of overhead.

Where an enterprise would own everything on any server that was placed on their networks, typically, a cloud provider uses something called *multitenancy*. Think of every computer system within the provider's network space as an apartment building. Each apartment in the building is rented out by someone else. The same is true with cloud computing services. There may be multiple virtual systems or applications within every physical server, just as an apartment building has multiple apartments and dwellers. Just as with an apartment building, everyone wants their own space. Sure, there may be some shared spaces (say, the management interface for example), but each apartment should be private so the people who live there can feel comfortable that their stuff is safe and no one else can get to it without permission.

This multitenancy is what makes cloud computing work. Multitenancy is what gives us economies of scale that allows providers to be in business. If it required a single server for every service being offered to every customer, then there isn't a lot of advantage to cloud providers being in business. There isn't much in the way of profit incentive. Once we open the door to multitenancy, though, a whole lot of other possibilities become available. In addition to dramatically reducing the cost of any individual application or server, we get a lot of other benefits as well.

The advantage to using a cloud provider is often thought of as less cost and less overhead from not needing floor space or air conditioning or power. More importantly, you get world-class uptime, which is hard for businesses to achieve individually. There was a time when telecommunications providers put a lot of store in what was called *five 9s* reliability. This meant 99.999 percent uptime. In practical terms, this translated to five minutes of downtime a year. If you think about a single power outage, for instance, would it be likely to last only five minutes? What about losing your Internet connection? More than five minutes? Of course.

This may not be a requirement for most businesses since a little downtime isn't usually the end of the world, but if you start looking at the numbers, you start to see significant changes. Once we get to four 9s, which is 99.99 percent, that's 52 minutes over the year. Again, most businesses could probably withstand the better part of an hour of downtime over a year without any ill effects. Three 9s, 99.9 percent, brings us to eight hours. If this is minutes spread over a year, it's not such a big thing probably, but if it's sustained in hours-long chunks, that may be more serious. However, it's not the sort of calculation that businesses often make—the probability of downtime and the cost of the downtime, weighed against

the cost of what it takes to maintain high availability. Even if they do, high availability is expensive.

Cloud providers, though, are in the availability business. If they aren't available to their customers whenever the customers need them, they aren't valuable to those customers. This is not to say all customers, since every business may have different needs, so every minute of every hour of every day needs to be covered since there is no way of knowing when a customer may try to get access to a service being hosted with the cloud provider.

Cloud providers can spend the money on high availability and fault tolerance. This not only includes power but also network and even computing infrastructure. After all, systems fail sometimes. Building in fault tolerance and reliability across the board from the physical layer all the way up to the application is hard work and, as noted previously, very expensive. This is another reason why going with cloud providers has some benefit. Not only can you save costs on your computing infrastructure and also get rapid turnaround on getting services spun up, but you also get a lot of other benefits that you wouldn't otherwise get.

Cloud Services

Saying “in the cloud” isn't very specific for several reasons. First, there is no actual cloud. What you mean is making use of a service provider. Beyond that, though, it doesn't describe what you are trying to accomplish by using the provider. While you are using computing resources with the provider, there are some categories that are clearer about what types of computing resources you are using.

At the lowest level is infrastructure as a service (IaaS). This is essentially acquiring systems. They are virtual systems, but you have access to a virtual device that will have the hardware specifications you provide when you provision the system. Figure 15.3 shows the selection of virtual machine hardware specifications using Amazon Web Services (AWS). In this case, you select the operating system before specifying the hardware, but using IaaS, you have access to the hardware, which will have an operating system installed on it. You don't have to worry about providing the operating system or even doing much in the way of configuration of the operating system. The provider takes care of giving you a base installation that is typically reasonably well-hardened. Unnecessary services have been removed, and you will only have the user account that has initially been provisioned to give you access.

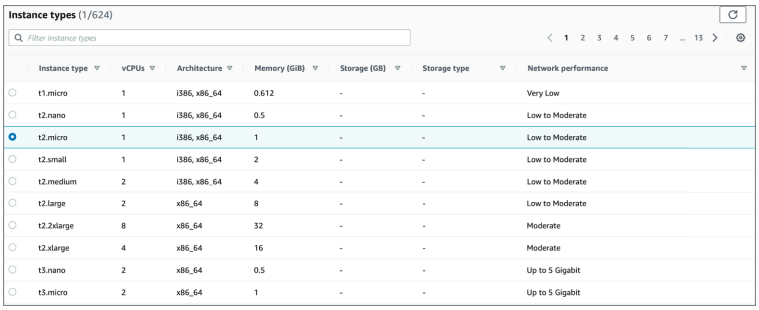

FIGURE 15.3 Amazon Web Services EC2 instance selection

This gives you the most control over the system, but it also means you have a lot more that you need to take care of. If you want an application installed, or any other configuration, you will need to take care of that yourself. However, you can select platform as a service (PaaS). In some cases, you may be developing web applications. This requires an application stack. This is traditionally composed of the operating system, potentially a web server and an application server that you can use to write your application. In some cases, this may be a Java-based application server, if you are using Java to write your application. Application servers like JBoss and Tomcat may be common. They take care of all the language pieces needed to run your application and also handle a lot of the underlying processing needed to handle a web-based application, which might mean exposing network listeners to take requests in. The application server will take care of basic processing of the request before handing the application-focused pieces into your code.

In the case of PaaS, you take care of your application, and the service you are buying will take care of everything else. You don't need to install any additional applications or configurations on the system you are getting. You just deploy your application. Figure 15.4 shows the selection of a .NET server in Microsoft Azure that can be used to deploy C# or Visual Basic applications. As with the selection of an IaaS service in AWS, you select the size of the virtual hardware and Azure will take care of deploying the system with a preconfigured .NET server installation. In Azure, you create a web app and you will get prompted to create a virtual machine with an application stack. You don't have to do any of the initial installation and configuration that would be common with an application server, which is great if you are a developer and don't know about configuration and management of the application or the operating system.

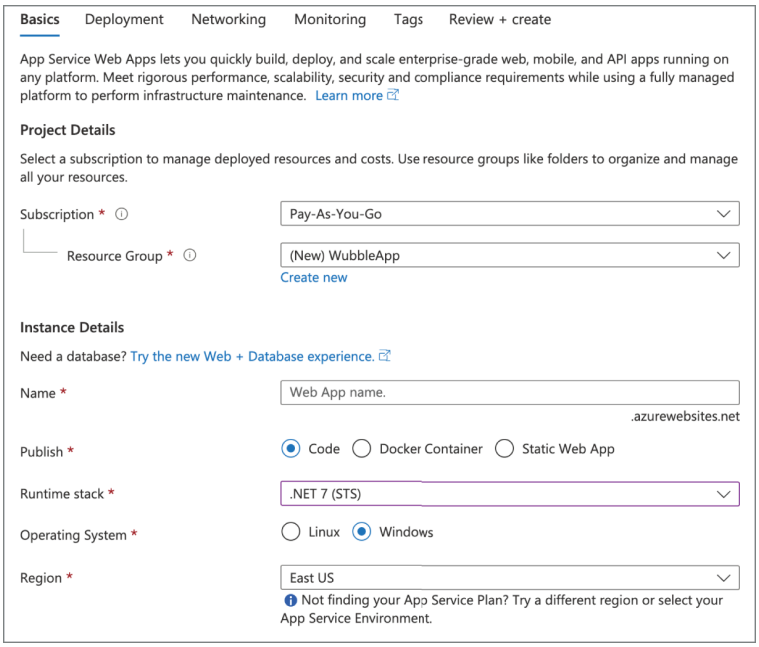

FIGURE 15.4 Microsoft Azure .NET server selection

Continuing up and away from complete control, we move to software as a service (SaaS). This has become a common practice. You may well use SaaS on a regular basis, especially if you are employed as a knowledge worker. A common business application is customer relationship management (CRM). If you have to do any sort of work with sales teams or customers, you may be familiar with CRM tools. The dominant CRM at the moment is Salesforce. This is an application that you would most commonly use through a web interface. Salesforce, the company, takes care of the hardware and software for you, so all you need to be concerned with is the data you are entering into the application.

There are a lot of web applications where the provider takes care of the software for you, presenting it in a web interface to make delivery a lot easier. You can, for example, easily develop diagrams using a number of SaaS providers like Lucidchart, which is shown in Figure 15.5. This can get you away from potentially expensive diagramming software like OmniGraffle or Visio. With some SaaS solutions, you can even create a free account and use at least limited capabilities of the software. With Lucidchart on their site, for example, you could create a limited number of diagrams, though you have access to all of the shapes and features of the software for any individual diagram.

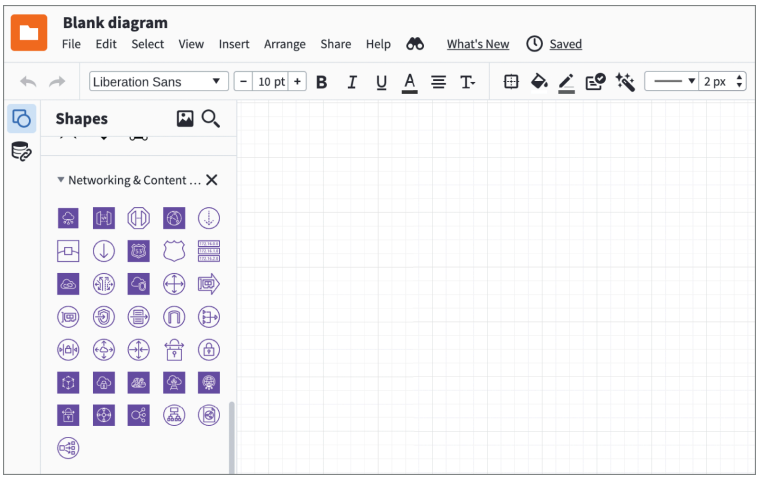

FIGURE 15.5 Lucidchart web interface

One of the issues with any of these providers is you are providing a lot of data that is stored with the provider, even if it's only user credentials. This leaks data into the provider, and that data is exposed if the provider is ever compromised. Some of these providers will use common, shared authentication providers like Google and Facebook. This means you would have to go after the authentication provider to gain access to someone's user account with a SaaS solution. However, in some cases, the provider may make use of these third-party authentication mechanisms while also allowing users to create accounts directly with the provider. As authentication is a complex process, if the provider has not implemented strong mechanisms to handle the authentication, the provider may be vulnerable to having the user accounts compromised.

Perhaps confusingly, we have another SaaS: storage as a solution. Most commonly for most people, this means Google Drive, Microsoft OneDrive, or Apple's iCloud, where you have some fixed amount of storage with a cloud provider. Just as with other cloud services, you would commonly get access to your storage through a web interface. However, in some cases, you can download an application to install on your computer if you'd rather interact with your storage as though it were just part of your file system. Figure 15.6 shows the OneDrive application running on a macOS system. This can be convenient for easily working on the same files from multiple systems. It's an easy way to get data from one system to another.

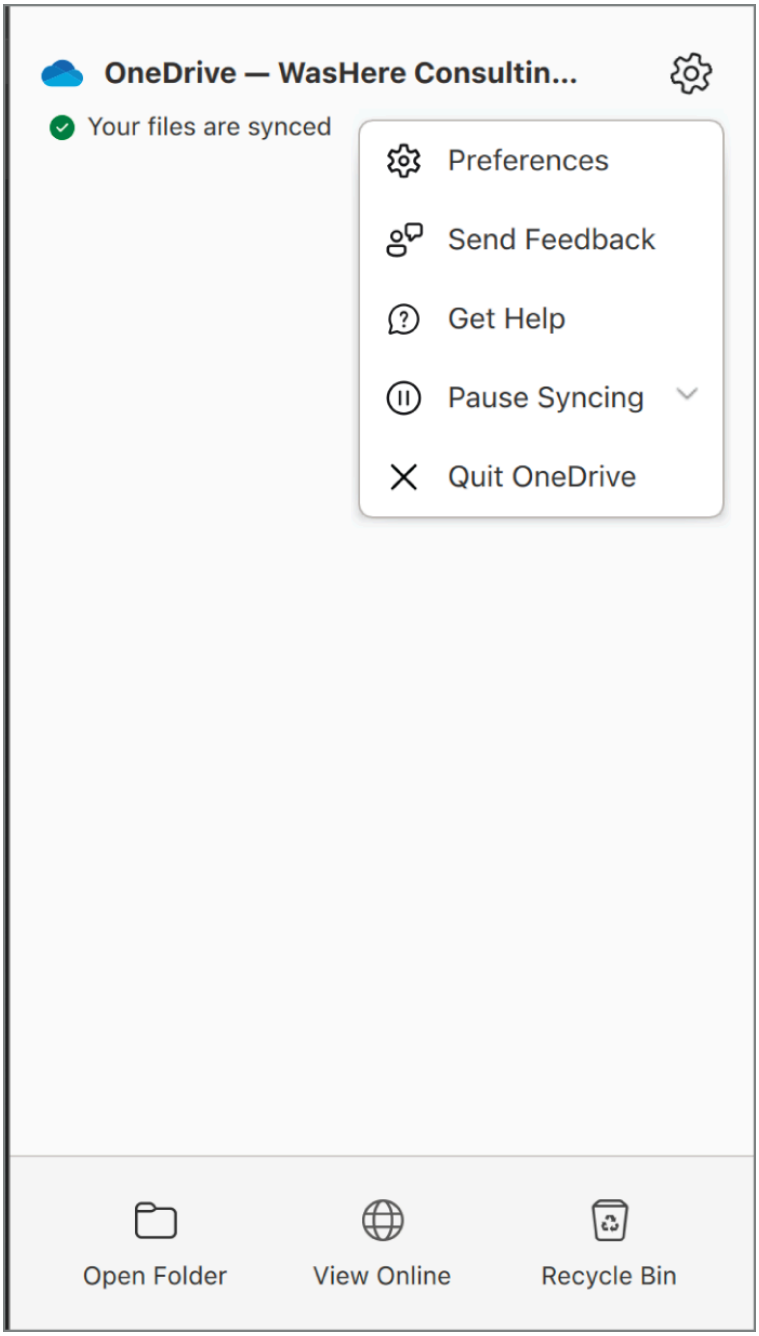

FIGURE 15.6 OneDrive application

However, that's not the only kind of storage you can get from a cloud provider. Businesses may want other types of storage. In a traditional application, they may want to store structured data, which would go into a database. However, not all web applications are created equal. Cloud providers like Microsoft and Amazon offer something called *blob storage*, where you can store unstructured data. Using blob storage, you can keep lots of different types of data without having to be concerned with the type of data it may be. Blob storage may be used for maintaining log information; files that may be served up through a web interface, images, video, or audio for streaming; or backup data. Whatever data you may have, you can store in blob storage. In AWS, these are called *S3 buckets*. You can see the configuration settings for the creation of an S3 bucket in Figure 15.7.

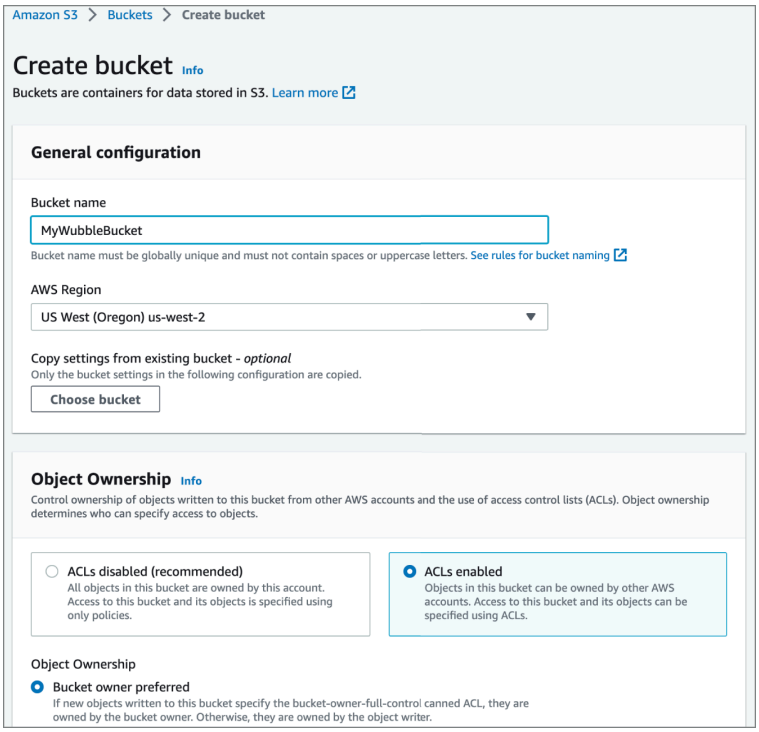

FIGURE 15.7 S3 bucket configuration

In the configuration shown, the public access is blocked, which is a reasonable default in most cases. However, if you want to provide access to users, you may need to loosen the restrictions to allow some level of public access. Once the bucket has public access granted, it becomes necessary to monitor files or other data that has been put into the bucket. Any data that is placed, even inadvertently, into a public bucket can be accessed by anyone who can find the bucket.

Shared Responsibility Model

Unlike servers and services on premise, where the organization is responsible for everything from physical infrastructure like power and real estate to the data and application, organizations will have a varying degree of responsibility depending on the service they select. Regardless of the offering selected, the provider will always be responsible for all of the physical aspects of service. They have to take care of power as well as heating, ventilation, and cooling (HVAC). They also have control of the physical systems and all of the virtual services that run inside. This means they take care of physical security by restricting access to the data center where the servers are housed only to personnel who need to have access.

When it comes to IaaS, the customer starts to take responsibility for the operating system. The provider will take care of having “systems” available, but all maintenance and configuration of the operating system and anything installed on the operating system will have to be taken care of by the customer. From a security perspective, this means the customer is on the hook to configure systems in a way that doesn't introduce vulnerabilities or remove necessary access controls that were established by default. It's also on the customer to ensure all necessary network controls are put in place to protect the server from anyone on the Internet.

Additionally, the customer is responsible for all identity and access management (IAM) for the server. The customer may need to control their own IAM solution to be able to create the users necessary to maintain the server and provide restricted access to additional users, which may include consumers of whatever service the company is offering. In some cases, a customer may make use of an Active Directory server to maintain users and roles that can be used to provide appropriate access to users assigned to those roles.

As we move up to PaaS, the operating system up through the application all become the responsibility of the cloud provider. This means they keep the operating system up-to-date with fixes. They may also have user accounts within the operating system so they can perform routine maintenance on the system as needed. They will also take care of the application server being used. In the case of something like a .NET server on Windows, that's part of the operating system in the sense that you can install the function from the operating system installer without needing an outside package from a third party. In the case of a Java application server, as another example, the provider will still take care of all updates as needed. The customer will be responsible for their own application, developed against the application server. The customer will also need to be responsible for any network controls necessary to protect the platform server.

When it comes to either of the SaaS services, the customer is responsible for the data. They don't have to be concerned with any maintenance of any system or service. There are no applications they need to keep up-to-date or perform any system administration functions for. The only administrative task they may need to be concerned with is user management. In the case of software as a service, you would typically need to provision users to get access to the software under the agreement with the business. If you were an individual user making use of something like Smartsheet or Lucidchart or one of countless other SaaS solutions available, you don't worry about any account other than your own. In either case, though, you create the user accounts and maybe assign specific permissions in the case of an enterprise-level agreement. Typically, you wouldn't have to be concerned with creating or maintaining an identity and access solution yourself.

In some cases, a business may use their existing identity infrastructure or directory services to provide authentication to cloud services. This may result from the implementation of federation services, which allows for a single repository of authentication information that several applications can use. These are special services because they allow applications outside of a single trust boundary to make use of this identity or authentication information. Figure 15.8 shows a diagram of what this may look like. Inside the cloud is an enterprise network where there is an Active Directory instance. All of the user information, meaning usernames, passwords, and roles/groups, is stored there. Outside of the enterprise are some additional services that have connections into the central repository of authentication information. Users can have a single user account that

can be used across services both inside and outside the enterprise, making it all appear seamless to them.

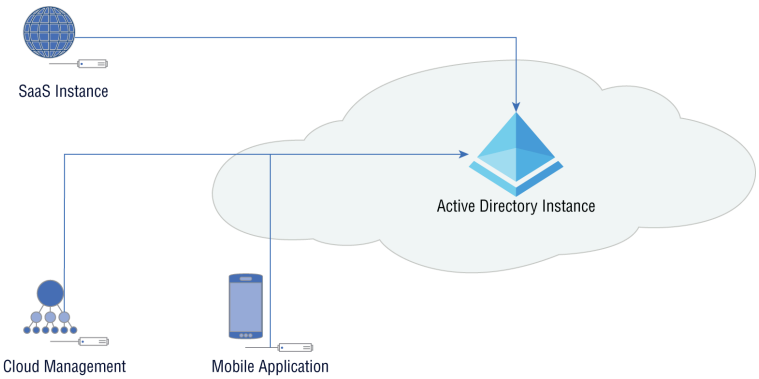

FIGURE 15.8 Active Directory federation

In practice, this means outside entities either need to have a way of reaching into the enterprise to perform the authentication in real time or need to have a way to synchronize the authentication details between the internal Active Directory instance and an authentication repository outside the enterprise. If you were using Microsoft Azure for your cloud services, for instance, you could create an instance of Azure Active Directory to perform authentication of your cloud services but then federate with your internal Active Directory, so accounts are replicated in both instances. All account creations, password changes, or role changes would happen in both instances, though there may be a lag between the two depending on the federation agreement in place.

Public vs. Private Cloud

All the providers we have been talking about are public cloud instances because anyone, anywhere can get access to them. The only restrictions in place are entirely based in the network design and architecture created for the application or service being hosted with the cloud provider. This is not the only type of cloud, however. You don't need to completely outsource to take advantage of cloud services. Keep in mind the essential characteristics of cloud-based services. Cloud services are on-demand and self-service, meaning you don't need to go to your information technology administrator, asking to have a system installed with services on it. The on-demand, self-service nature is supported by easy access to a management portal, commonly provided over web-based protocols like the HyperText Transfer Protocol Secure (HTTPS).

Additionally, cloud services are multitenant. They can be multitenant because of resource sharing. You have systems that are far more capable than a single system instance requires. Because of that, you can have multiple service instances, whether they are virtual machines or virtualized applications inside containers, on a single physical system. In a public cloud environment, the multiple tenants may be across multiple businesses or individuals. There is no guarantee how the services are going to be provisioned in a public cloud environment.

There are private clouds, though. A private cloud has the same functionality as a public cloud provider may have, at least in terms of on-demand and self-service as well as being multitenant. In a private cloud, multitenant doesn't mean multiple businesses on the same server. It may mean multiple divisions within the same business on the same server. A private cloud can offer the same automation features a public cloud does, including the ability to use templates for rapid provisioning of services, as well as using code (software or configuration files) to automate the provisioning of systems and services.

One commonly used set of software for private cloud implementations is OpenStack. Using OpenStack, you would have multiple systems in your enterprise that could be used to manage virtual machine instances as well as virtualized application instances. The OpenStack architecture breaks out multiple services that could be spread across different machines for storage management, user interface (a web portal for self-service and administration), compute and containers services, and networking and load balancing. Everything you need to support your own cloud implementation is available in OpenStack. This allows you to better control access to these services, because they would be inside your enterprise network where you can use network- as well as user-level access controls to restrict who gets access to these services.

Grid Computing

While computing capabilities have increased significantly over several decades, there are still problems that require much more processing capabilities than even modern processors may be capable of in a reasonable amount of time. A common approach to adding more processing power, even on single systems, has been to throw more processors at the problem. Modern processors have multiple processing units all on a single chip. In some cases, you may even find systems with multiple physical processors. There was a time when this was an approach to something called *supercomputers*, which had large processing capabilities, unseen in systems used by normal users or businesses.

Grid computing is a way of increasing the processing power being thrown at problems, though it does require the problem to be approached in different ways. When you are tackling a problem like cooking dinner, for example, you probably have a set of steps that are done one at a time in a linear fashion. You may be able to go faster if there were tasks that could be done at the same time if there were multiple people in the kitchen. However, very often we as humans tend to think about linear, procedural approaches to problem-solving. When you have multiple processors, you have to think about how you are going to break up the solution into a way for multiple processing units to tackle parts of the problem at the same time. This can lead to challenges with coordination and communication, since the different processing units may need to be highly aware of what other processors are doing since each processing unit may need data being handled somewhere else.

Of course, it's also possible to just take a large number of cases of a problem and spread those cases out over multiple systems. This also requires

coordination and communication since each individual processing unit has to be in touch with a controller to indicate progress, asking for more workloads as individual problems are resolved. Commonly, this requires networking capabilities and communications between all the different systems. From a vulnerabilities and attack perspective, this means there are services listening and communicating.

You may have heard of or run across grid computing projects. In the 1990s, distributed.net was a project started to take on a cryptographic challenge from RSA Lab. Since that time, the project has handled several other cryptographic challenges from RSA Lab. SETI@home is a grid computing initiative meant to search for extraterrestrial life from radio signals in space. These are both projects that use a technique called CPU scavenging, meaning they make use of unused processor cycles on computers. These are systems that may be used by anyone for their normal, everyday computer needs. When there are processor cycles that are going unused, the software performing these tasks will be allowed to use the processor. Another CPU scavenging project is Folding@home, which is a project that has a goal of helping scientists identify treatments for diseases by simulating proteins. As noted earlier, all of these projects need to engage with other computers to coordinate their work. Here is an example of a Folding@home process listening for requests on a Linux system:

```
FAHClient 433209           root    7u  IPv4 19284889      0t0  TCP *:7396 (LISTEN)
FAHClient 433209           root    8u  Ipv4 19284890      0t0  TCP *:36330 (LISTEN)
```

Figure 15.9 shows a simple grid computing setup that would work for the types of projects we have been discussing. This has a controller that manages the different clients doing the work. Botnets are also examples of grid computing. At the top of the diagram, you would have the controller, but at the bottom, you would have the bots. They are still technically clients, but the difference between a botnet and one of the projects previously mentioned is with a botnet, the owner of the computer is not aware they are sharing the use of their system with someone else.

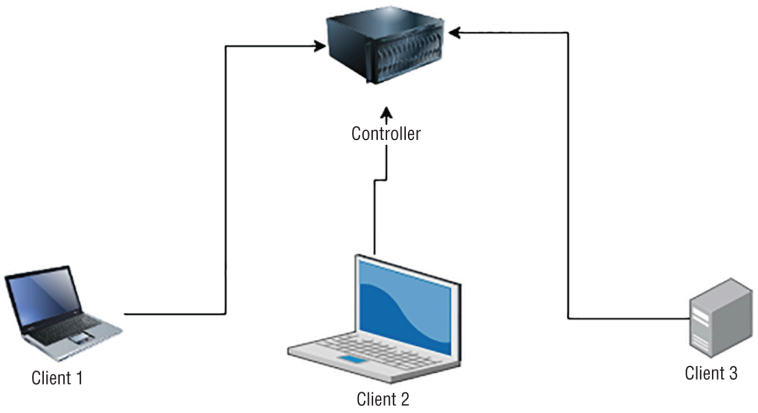

FIGURE 15.9 Creating a function app in Azure

Attackers have also used grid computing for tasks like cryptocurrency mining. Once a system has been compromised, the attacker may install mining software onto the victim computer. The mining activities are associated with a cryptocurrency wallet that belongs to the attacker, so the at-

tacker benefits from the activities of these client systems they have compromised. As many of these tasks, like anything to do with cryptography or even protein folding, tend to be math intensive, a computer that has a graphics processing unit (GPU), which is geared toward math-intensive tasks, is even better suited. If you were to look at statistics for some of these projects, you would see references to the use of GPUs. Systems with powerful GPUs would likely be far more effective at contributing to these sorts of projects than general-purpose processors.

Cloud Architectures and Deployment

In many cases, you might just take the systems you have deployed on premise and move them just as they are, in terms of configurations, to a cloud provider. This is what is sometimes called a *lift-and-shift approach*. You are taking your infrastructure just as it is and placing it into someone else's network. The only thing that has changed, aside from network addressing, is the physical location of the systems. You still have all the potential problems that come with having systems under your control. You need to make sure the operating systems and applications are kept up-to-date. You still need system administration staff to support the systems and applications.

Migrating to a cloud environment may mean that you can get rid of some of that overhead. As mentioned previously, you can move your Active Directory instance to Microsoft Azure. That removes all the maintenance necessary on a number of servers. All you need to do is take care of the user management and you don't have to worry about the underlying operating system and the updates necessary to it. This works well if you have also outsourced your email to Microsoft using Office 365 (O365). Everything is stored with Microsoft, and you manage it all through web interfaces, which means you can get away without using desktop systems for management in addition to not needing your server infrastructure. This is not to say you won't use desktops for other purposes than managing your Microsoft 365 (M365, which is O365 plus Azure) infrastructure.

You are not bound to use the same old deployment models for your infrastructure, though. First, you don't need to use on-premise instances of all your server-based applications that your users are using on a regular basis. Traditional business applications like those required for human resources, billing, recruiting, and other essential business functions may have software-as-a-service instances, which means you can sign up with the provider directly, having them host the software for you. Based on this, you don't need to have servers around for that software. This reduces the attack surface within your network because there aren't operating systems and services sitting around for attackers to compromise.

Even in cases where you are building your own applications, you can think about different approaches to the way you design them. In a traditional, on-premise environment you may be constrained to using virtual machines or even containers because doing something more complex is just too…well, complex. In reality, when it comes to using a cloud provider, you can simplify a lot. It's just that the implementation to get to

that simplicity is hard, or at least takes a lot of work. If you are writing a program, do you want to think about the system implementation, or do you want to think about the program? Why should it be any different when it comes to writing web-based applications?

Programs are generally decomposed into functions, sometimes called *procedures* or *methods*. This provides the ability to reuse functionality over and over. It also provides the ability to be reactive, meaning you can call a function when the program needs it in response to something else that happened in the program flow, such as input the user has provided, or a button the user clicked. Without these functions or methods, either you would have a single path through the program or you'd end up with spaghetti code with no structure and a lot of jumping around in an undisciplined manner. This makes it difficult to follow the code or fix problems that might arise.

Now that we have our functions, we can just focus on creating the functions and let something else take care of the program flow and calling the functions into being. This is where it gets complex. Normally, you'd need to have determined where the function was going to run ahead of time. You'd need a virtual machine or a container to house the function because you need an operating system to take care of getting the functions into the processor to run, then unloading them (or replacing them in the processor) when they have run to completion. Normally, this requires a single operating system because you have data that you are passing back and forth between the different functions. You can do this with remote code execution mechanisms like remote procedure calls or remote method invocation, but typically there would be a central process that takes care of knowing where the entire program is and the state of all the data.

The Web is a stateless being, meaning every request has to start from scratch because HTTP, the protocol used to transmit data back and forth between a web server and a client like a browser or mobile application, is stateless. HTTP has no ability to keep track of a user's state. It only knows what has been sent to the server with that request. As that's the case, the piece that knows where everything is resides on the client side. This means we can take advantage of the client knowing the user state to allow the server to really not know anything except what gets handed to it. This helps open the door to serverless processing. We can truly write functions that are called as they are needed without having to know where the function is running.

All of this is a long way of getting around to serverless functions. Cloud-native designs will often use serverless functions. You don't provision a complete virtual machine or even a container that houses a virtualized application. You write a function and let your cloud provider take care of how your function will be executed and when it will get called. In AWS, you use Lambda, which is serverless computing. In Microsoft Azure, you create a function app, as you can see in Figure 15.10. This configuration page allows you to specify your runtime stack, which is the language that will be used to process the function you write. Using this method of creat-

ing applications, we have gone from physical systems to virtual systems to virtual applications to virtual functions.

There are two important ideas to note here. The first is there is no landing space for an attacker to get to in the absence of vulnerabilities in the implementation of serverless functions. Of course, in reality there is a server under the function, or else there would be nowhere for the function to run. Depending on the implementation, there may be a surface area for the attacker to get access to through a vulnerability in your code. This relies on an implementation where some additional external program is available in the virtualized system or container where the function is executing. It also relies on something that shouldn't be happening, which is that the virtual implementation, whether container or server, remains running after the function has ceased. Because they are small and simple by nature, these serverless functions should be easy and quick to instantiate and then tear down on an as-needed basis, making them ephemeral.

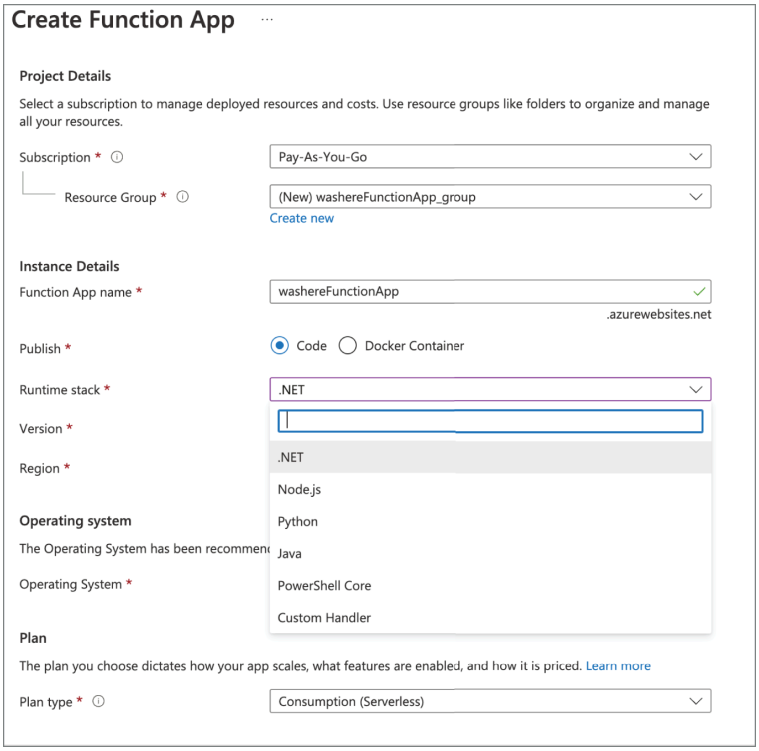

FIGURE 15.10 Creating a function app in Azure

MAINTAINING ACCESS

It's theoretically possible that a vulnerability in the function's code could leave the function functional, meaning still executing in memory, even after the attacker has taken control of the flow of execution. This would mean the implementation isn't at fault for not tearing the virtualized instance down when it was complete. However, the use of virtualized functions does have the potential of significantly reducing the attack surface by leaving an attacker with no firm footing to remain in.

Responsive Design

The smaller and lighter weight an implementation of an application component, the faster it can be to create an instance of that component. This means you can create an implementation of an application where the instances of each component exist for only as long as it takes to serve the request being made. When you are paying for computing on an as-used basis, having the component run only when it is being used can save on costs. Creating applications where computing resources are used only as needed is more efficient from a cost perspective.

Applications can be designed to be responsive to demand. In a traditional application deployment, especially one before we started to use virtualization technologies, you had a load balancer out in front to manage the requests coming in. The load balancer would then spread the load out across multiple servers behind. If you wanted to handle more requests, you created a new instance of the same server and added it to the load balancer configuration. If you had some foresight, you would have built these additional systems ahead of time and maybe left them powered off until they were needed, to save costs on power and cooling. When you were ready and needed them, you could power them up and adjust the load balancer configuration.

Today, we have the capability, with both system and application virtualization, to adjust to demand on the fly. As requests come in, the needed systems or application instances (containers) get created and added to the pool of computing resources available. This is especially true with the use of infrastructure as code (IaC). Using IaC, you determine exactly what a system is going to look like: the operating system, the applications, and the configuration settings. A decade and a half or more ago, having system deployments that were this planned out was harder. You could create a system installer, but you'd constantly have to be updating it, and it would act like a system installer; you would install the system from the operating system up, though you might have additional elements that got installed as part of the process.

From a security perspective, the use of a responsive design, where individual components may become available on the fly and then, very shortly after, disappear, has some advantages. If you, as an attacker, are able to gain access to one of these resources, you have no idea how long that resource will be available. You could just be getting going on gathering credentials and trying to move to other connected systems when the resource disappears. Additionally, as so much of this style of application development is virtualized, you get access to only a small piece of a system. In the case where containers are used, you may not have any place to land since most pieces of a traditional operating environment are not in the container.

Cloud-Native Design

In addition to the ephemeral nature of cloud deployments, where individual elements will disappear after a short time of being available, there are some elements of cloud-native design that are very different from a

traditional application design. A traditional application is monolithic, meaning everything is contained within a single executable or at least within a single execution space. Cloud-native design decentralizes everything. Rather than calling different functions within the same process space, cloud-native design may be more likely to use a service-oriented approach where the functions that may get called are broken out into separate execution spaces all to themselves. These are then called *services*. You can see a simple example of what this might look like in Figure 15.11.

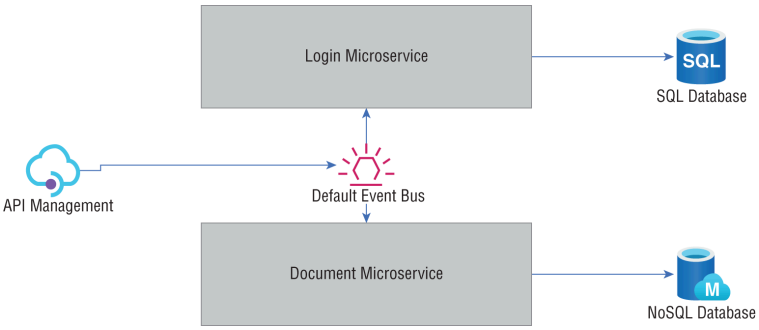

FIGURE 15.11 Microservices design

The microservice architecture moves away from a monolithic application, which means any service can be implemented in any way that may be beneficial. These services can be implemented in virtual machines, but that increases the attack surface because there is a whole operating system that can be used by the attacker. Services may be implemented using containers, so small programs can be virtualized. This reduces the attack surface, but there is still the possibility of making use of the mechanisms used to manage the container. In some cases, containers may expose ports. Communications to that port will be passed into the container. This may mean the virtualized application receives the communication, or it could be something else receiving the messages. This may include allowing shell access to the container image.

Another approach includes the use of a serverless design. This may mean simply using functions rather than virtualized applications. There is no application. There is no container. There is no operating system. There is a function. This doesn't remove the potential for the overall application to be compromised. Remember, the goal of an attacker isn't always, or even usually, shell access, though that's a common misconception. It's the way attackers have historically worked (historically, in the sense of it being a while ago). Today, attackers may simply be looking for data they can sell. This means access to the data, whether unauthorized by using an application vulnerability or authorized through stolen credentials. Functions don't prevent that sort of attack. Functions can make it much harder to gain system-level compromise, but not application or data access.

Serverless functions are event-driven, meaning there isn't a single application that determines whether a function gets called. Instead, getting a function called may be easier to trigger because it will depend on events. One of these events may be because the overall application uses a publish/subscribe model. There is an event bus that runs throughout the

application. Functions send messages to the bus by publishing them. Other functions can receive messages by subscribing to them. If a message that one of the serverless functions subscribes to gets put on the bus, the function will get called. Another way to get a serverless function called may be through the use of an HTTP method, like GET, POST, PUT, or DELETE. A serverless function may be called when the overall web application gets one of these HTTP requests. Additionally, the existence of some specific HTTP headers may cause functions to get called.

Since there may be a lot of services running, each of them capable of publishing methods onto the message bus, there could be a lot of different ways of getting these functions called. Effectively, the attack surface area has increased because there may not be a single-entry point to potentially vulnerable code. These multiple entry points can also end up causing issues with authentication. Rather than relying on the overall application to perform authentication, each function will have to expect that there may be a way for a request to get to that function without having had any authentication.

While serverless functions may not give an attacker easy access to an underlying operating system, the OWASP Top 10 list of vulnerabilities still applies. This is especially true for vulnerabilities like those from misconfiguration or weak third-party components. Sensitive data can still be exposed even when using cloud-native design, microservices, and serverless infrastructure.

Deployment

Deployment today is commonly automated. It doesn't matter what the target is, whether it's virtual machines on something like a VMware ESXi server or a Microsoft Hyper-V server or it's containers with a cloud provider, the deployment of system assets can be automated. There are different approaches to this automation, though a common one is to use an orchestration platform. Using orchestration systems is commonly called *infrastructure as code* (IaC) because deployments are done either using a scripting language or using client-server software and configuration files that define the deployment.

A common software package to use for IaC is Ansible. Ansible is a software package that uses a server where the configuration file resides and then a software agent that may be used to do additional configuration from inside a running system if that's necessary. In the following example, Ansible will create a new instance of an existing template on a VMware ESXi server. The Ansible server software will contact the VMware server using the credentials provided at the address specified and send commands to the server to get the template cloned, creating a new instance of a web server. The configuration file shown here uses the Yet Another Markup Language (YAML) format. This is common with other, similar software packages like Salt. While this example shows the creation of a virtual machine on a VMware server, you can perform similar functions with any cloud provider.

Salt Configuration

```
- name: Clone existing template
  hosts: localhost
  connection: local
  gather_facts: no
  tasks:
  - name: Create new instance of web server
    vmware_guest:
      hostname: 192.168.3.5
      username: root@192.168.3.5
      password: P4$$w0rd!
      validate_certs: False
      name: webserver_vm
      template: webserver_template
      datacenter: DC1
      folder: /VMs/templates
      state: poweredon
      wait_for_ip_address: yes
```

Another approach to creating instances of any virtual resource is to use a scripting language. One of these, Terraform, is a language that is used for the purpose of building environments in code. You may get that from the name. Terraform is a word that means transforming an environment. It's commonly used in science fiction when making an existing planet, elsewhere in the universe, into something resembling Earth, or at least making it more habitable for humans. However, you don't have to make use of a specific language like Terraform. You can also make use of a general-purpose language like PowerShell. As an example, the following code will create an instance of a virtual machine within Microsoft Azure:

```
$resourceGroup = "OurGroup"
$location = "westus"
$adminName = "powershell-user"
$adminPass = "P4$$w0rd!"
$dnsName = Read-Host -Prompt "DNS name for the IP"

New-AzResourceGroup -Name $resourceGroup -Location "$location"
New-AzResourceGroupDeployment `
    -ResourceGroupName $resourceGroup `
    -TemplateUri "https://myhost.onmicrosoft.com/azuretemplate.json" `
    -adminUsername $adminUser`
    -adminPassword $adminPass `
    -dnsLabelPrefix $dnsName

 (Get-AzVm -ResourceGroupName $resourceGroup).name
```

This code example requires the use of the Azure PowerShell module. You can also run this code from an Azure Cloud shell, which gives you a PowerShell interface to send programmatic commands to Azure. This is not to say that you have to use Azure to be able to perform programmatic functions. The other large cloud providers, Amazon and Google, have similar capabilities, including command-line interfaces to their application programming interfaces (APIs). These are software packages you can install on your local machine that will allow you to use commands, which could be scripted, to send directly to the cloud provider of your choice.

What you may have noticed in the PowerShell example is that there is a reference to a template. This is a configuration that can be created in Azure and stored using JavaScript Object Notation (JSON). What is referenced in the script is a JSON file that could have been created when building a virtual machine in Azure. Once you are done configuring all the settings you want, including the disk and memory size as well as the operating system you want to use, you can store your configuration as a JSON file for later use to get the same settings every time you want a new instance of the system.

Other cloud providers will also allow you to create templates that can be reused, though they may use different techniques to get there. Amazon Web Services (AWS), for instance, uses something called CloudFormation Designer to design a complete network, if that's what you are looking for. Figure 15.12 shows the CloudFormation Designer. This works similarly to diagramming tools that you may have used. You find the element you want, dragging it out of the toolbox on the left into the field. You can edit parameters for each element you are making use of. When you are done, you can store your completed template in either JSON or YAML format. Once you have a template file stored, you can reuse it to create any instance of the same design.

The advantage to using IaC is you get repeatable and consistent system and network implementations. Additionally, because they are configuration files or scripts, they can be tested to ensure you are getting what you expect. This is helpful if there are any changes to the templates or IaC configurations. Anyone in charge of operating a cloud environment will need to make sure that any change doesn't impact the operation of the overall application.

Dealing with REST

One of the problems with HTTP is it was never designed to maintain a connection between the client and server. Every time a client engages with the server, it's like it's the first time. The server, using just HTTP, can have no way of knowing who the client is because there is nothing in the HTTP protocol to allow the server to track any of that information. HTTP is considered a stateless protocol. The protocol itself is never aware of anything other than a single request and response. This makes writing applications difficult. Of course, HTTP was never designed originally for anything other than simple requests and responses. You need to remember it was developed as a way for academics to share papers and information among themselves, primarily. In the intervening decades since it was originally developed, the demands on HTTP have increased.

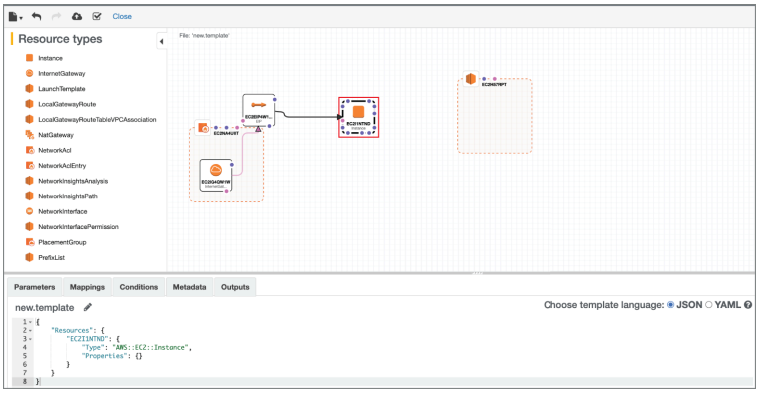

FIGURE 15.12 AWS CloudFormation Designer

Currently, a common approach to writing web-based applications, including those that may use an interface on a mobile device with processing done through web-based servers, is using APIs. The mobile device application, or even an application presented through a traditional web browser, will manage calling the API stored on the web servers. This kind of design may make use of Representational State Transfer (REST). Applications that make use of the REST design style are called RESTful. REST itself is not a protocol or even a specific way of implementing applications. Instead, a RESTful application makes use of a set of architectural properties. These properties include the following:

- **Client-server Architecture**: The application is implemented in a multitier fashion, with a client and a server, though in practice there may be additional elements necessary to support the functionality of the application.
- **Stateless**: There are two types of state that are necessary for the application to function. The first is the resource state, which the server knows about. The second is the application state, which the client knows about. The server knows only the details it has control of and needs to be told about anything else by the client. The reason RESTful applications are considered stateless is because there is no known state for each client that is maintained by the server. The server is told about the application state by the client.
- **Cacheability**: In some cases, there may be static data that is transmitted from the server to the client. Any data that doesn't rely on a programmatic exchange from the server, which could change from one request to another, could be maintained locally and presented as needed without having to have the data transmitted by the server again.
- **Layered System**: As there may be multiple elements involved with a RESTful application, layering is necessary. This allows for the client and the server to communicate to one another without having to be concerned with any other implementation details. The client doesn't need to know if there is a database behind the application. It is in a separate layer. Similarly, if there are load balancers in use, neither the client nor the server should need to know about them. This allows a nice modularity in design, so pieces can be swapped in and out without interfering with the functionality of the application.
- **Uniform Interface**: No matter what happens with the application on the server side, the client will use the same interface. This relies on

each request identifying the resource. The application on the client side needs to know what it needs and will call the right resource as needed. Data being transmitted between the client and the server should be self-descriptive since data may only be semistructured. Different data elements may not always be in place, so the elements that are sent need to be identified clearly so either party (client or server) can decipher correctly.

RESTful applications make use of common HTTP verbs—GET, POST, PUT, DELETE—to manipulate the state of the application and its underlying data. For the most part, communication between the client and server in a RESTful application will look just like any other HTTP communication. These are the same verbs, or requests, used for every request between a web browser and a web server, though PUT and DELETE are more likely to be used by an application than a request for static information. GET and POST, though, are going to be common for garden-variety web requests. The problem with testing RESTful applications is you need to know the endpoints used by the application. These endpoints are essentially function calls, though they will look like a web request. The endpoint is the path to the function, relative to the top of the web directory. Figure 15.13 shows an example of a web application. The exchange between the client and the server was captured using Burp Suite Community Edition.

What you will notice in Figure 15.13 is that the endpoint called on the server is `/sdk/`. This can be seen in the POST request on the left side of the screen capture. This is the location of the function that is being called. Whatever function is on the receiving end of this call needs to be able to handle the request coming in. The request itself is in the eXtensible Markup Language (XML), which is a self-describing data formatting language that HTML is based on. The request, as well as all of the information the server needs to handle the request, is passed from the client to the server. The response also includes an XML payload. The application running in the client browser needs to be able to update its representation based on the information coming back from the server.

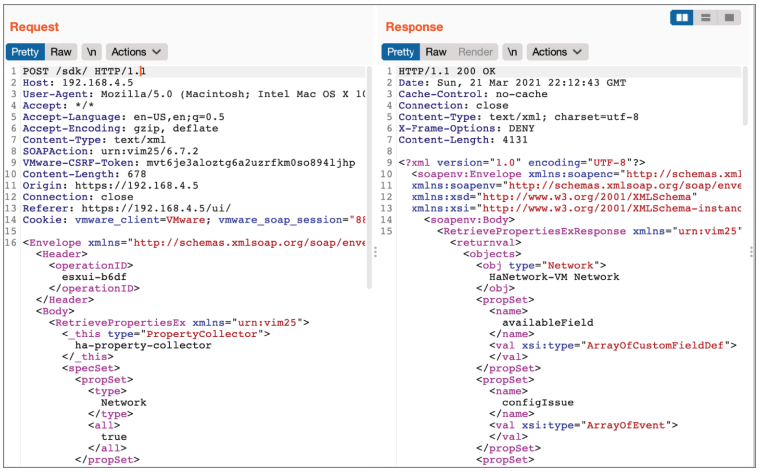

FIGURE 15.13 Burp Suite testing RESTful application

Once you know an endpoint, you can start testing it. In Burp Suite, this is an easy task. You select the request you want to use, meaning find a re-

quest in the client-server communication stream that targets an endpoint you want to do some testing against. You can send that request to the Intruder portion of Burp Suite. This will allow you to identify parameters you want to vary to see if you can break the application in some way. This may include altering data, getting unauthorized access, or causing the application to fail in some way. Figure 15.14 shows a request that has been sent to Intruder in Burp Suite. Burp Suite has automatically identified parameters that it believes can be modified. As an example, the following has been identified by Burp Suite as a parameter:

```
<operationID>§esxui-b6df§</operationID>
```

FIGURE 15.14 Burp Suite Intruder

The data between the two § characters is what Burp Suite believes can be adjusted. Using the Intruder function, Burp Suite will take a set of inputs and continue to make repeated requests with different values in the parameter that has been selected. You can create different types of data manipulations using Burp Suite. Figure 15.15 shows the different payload types Burp Suite allows. Perhaps the easiest is Simple List, which allows you to use a file of a list of values. This may require you to understand the parameter to know what may make a good value to provide to the application. If you don't know what type of data would be useful, you can use the Brute Forcer option. This allows you to specify a character set to generate values from. You tell Burp Suite the minimum size of the payload and the maximum size and it will try every possible value. Keep in mind that the number of requests will increase exponentially. Using lowercase letters and numbers, with a four-character payload, Burp Suite indicates that the number of potential payloads is more than 1.6 million. Longer payloads or more characters will significantly increase the number of requests.

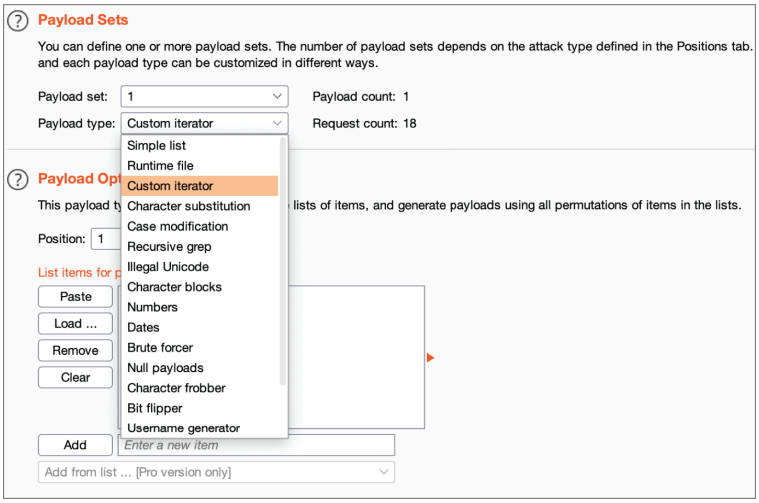

FIGURE 15.15 Payload options

One problem you may run into, though, is being able to identify the endpoints. This is a problem. There is no standard way of identifying all possible endpoints within an application. In the case of the VMware web interface, which we have been working with, there appears to be a single endpoint that all requests go through. This is not going to be common. An application may have dozens of endpoints, depending on the complexity of the application. To test the endpoints, there are a couple of approaches you can use. First, if you are doing white-box testing, you can get a list of all the endpoints from the company you are doing the testing for. If you are doing a black-box test, you can brute-force the endpoints. This is a technique called *forced browsing*. Essentially, you make a request to the web server using an endpoint name. If you receive a not found error, typically something in the 400 range, from the server, you know you haven't found an endpoint. Other responses may require parsing. A 200 response means you have found a name the web server knows unless the web server has been configured to return a 200 response but an error page. This is not common, because it violates the protocol, but it does happen.

There are different ways you can perform forced browsing. You could write your own tool using a scripting language if you had a set of wordlists, for instance. Making web requests and tracking the response is easy enough to do in a language like Python. Word lists are available around the Web, including a list of API endpoint names at https://github.com/chrislockard/api_wordlist. This wordlist could also be used in a tool like Burp Suite. You can use the Intruder function again, selecting the last part of the web request to vary. Additionally, you could use a tool like the Zed Attack Proxy (ZAP). This has a feature built in for forced browsing. You can select a site in ZAP, right-click to bring up the context menu, and then select Attack; from there you can force browse the site or the directory, for instance. You can see the context menu and flyout menu in Figure 15.16.

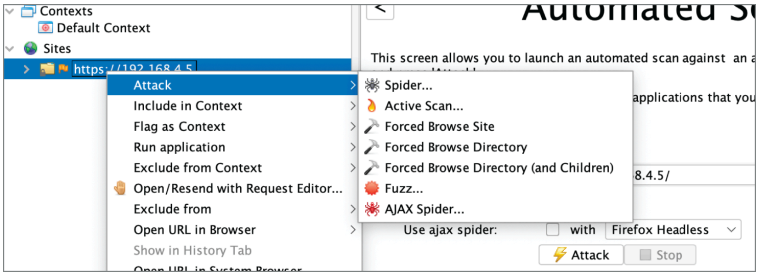

FIGURE 15.16 Force browse in ZAP

Once you have identified the endpoints that are available, though even force browsing is not guaranteed to turn up all endpoints, you can go back to testing in the same way you would test other web applications. After all, we are still just talking about a web application with code running on an application server. There may be nuances to testing different RESTful applications. For instance, you may need to vary the user agent string to test some applications. If there is an expectation that requests are coming from a mobile application, using the traditional IE, Chrome, or Firefox user agents may not work. The application may be looking for the right user agent to ensure the request is coming from the right place.

Common Cloud Threats

In practice, the use of cloud providers doesn't necessarily change the avenues attackers can or will use. Attackers can still use phishing attacks to gather credentials that can be used to gain access to a cloud environment. No matter where your resources are located, you need to do the blocking and tackling necessary to protect them. This may require understanding the different features of a cloud environment to perform all the right functions, but all the essentials still need to be in place. The following are some common threats that you may need to address if your resources are with a cloud provider.

Access Management

You are going to have users, even in a cloud environment. Commonly, you will have to deal with all the same identity and access management issues you are dealing with on premise. The same rules apply in a cloud environment as apply in your own network and systems. All access should be reviewed and approved to ensure that each user has only the level of access necessary to perform the job they have. This should mean using role-based access control rather than trying to manage assigning individual permissions to every user. Every user should have a strong password that isn't used in other locations. Additionally, multifactor authentication should always be used, regardless of where you are located.

Where it gets challenging for some people is not knowing where to locate the identity and access management feature in the different cloud environments. It's not the same as just loading up the Active Directory management application on a Windows server, unless you are using a federated identity solution between your cloud provider and your local Active Directory instance. This would allow you to use the same users on your network and in the cloud provider network. The credentials would be

validated against your local Active Directory instance, even though the request is being made against the cloud provider.

Typically, when you are managing access to your cloud environment, you would go to the console for your cloud provider and locate the module called identity and access management (IAM), or its closest variation. In Google Compute, it is called IAM, which you can see in Figure 15.17. Under Microsoft Azure, you would go to All Services and then Identity. AWS has a feature called Identity and Access Management. From any of those places you can create users and assign permissions. Once you have created users, they can be assigned to resources you have created within your cloud environment. Just as with any other identity solution, keep in mind that roles may carry across different systems. If you need to have a user be an administrator of a certain system but not any other, you may need to create different roles to accommodate that, rather than just making the user an administrator. Always carefully assign permissions to individual users.

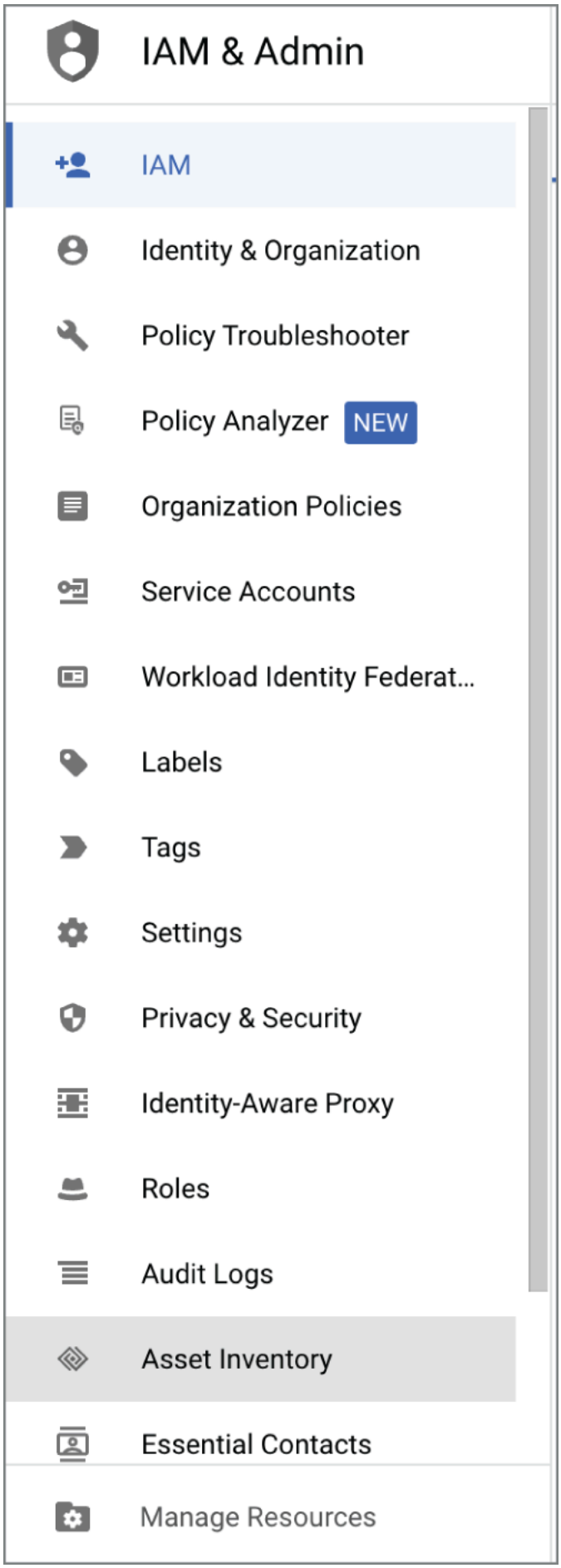

FIGURE 15.17 Google IAM

One issue to consider when it comes to access control is that you can create cryptographic keys to access individual resources. This is certainly true when it comes to AWS compute instances. If you create a key but fail to control access to the key both through the use of a strong passphrase on the key and not sharing the key and the passphrase, you take the chance of exposing access to the system the key is being used for. If the same key is used for multiple systems and that key gets out, any system the key is being used for will be accessible to anyone who has the key and the passphrase that is used to unlock the key.

One approach to protecting resources in the cloud is auditing and logging. You should be logging access to resources and monitoring that access. Attackers will use a variety of techniques to obtain credentialed access to resources. This may include trying known credentials. Without logging access, including the source of the login attempt, you won't know when an attacker logs in from a geographic location that shouldn't be possible. With cloud resources, there aren't always the same access restrictions as with on-premise systems where you may need physical access to facilities to be able to connect to the system. If you are connecting over the Internet to the cloud-based resource, the attacker can also connect in the same way you are.

Data Breach

The most important resource you are maintaining with a cloud provider, or anywhere you are keeping it, is data. Ultimately, you may not care if an attacker gets access to a stand-alone system with no data access. If they use your system for launching attacks, you may end up paying charges for network and computing resources. If you catch it in time, there is no lasting damage. Data is a different story. Data may include personally identifiable information, which could require notification of a data breach to the public if the data is stolen, not to mention a lot of time and energy on the part of the people whose data it is to recover from the breach. In the case of intellectual property, you may be exposing the business to long-term impact from competition in the marketplace.

There is more than one way to be exposed to a data breach. The first one is through application exposure, meaning an attacker compromises the application and gets access to the underlying data. The second is if the data is being stored in an unstructured way. In the case of AWS, for instance, you'd be storing it in an S3 bucket. By default, AWS buckets come with all public access turned off. Figure 15.18 shows the configuration of an S3 bucket with the protections turned off. This requires that not only the setting gets disabled but also that you check another box acknowledging that you know the problem with making an S3 bucket public.

This may be what you want. In some cases, you want users to be able to get access to data that you store with a cloud provider. The problem with this, though, is that it's possible for someone to accidentally store sensitive information in a public cloud storage instance. As soon as the data is in the cloud storage, if public access is allowed, anyone can get to it. Don't assume that just because a URL isn't known to the outside world, the data

can't be accessed. It should never be expected that hiding an instance by not publishing it anywhere is the same thing as protecting it.

You can help protect against data compromise by regularly assessing permissions on resources to ensure the right access controls are in place. Additionally, using a data loss prevention capability, whether from the cloud provider or a third party, can help protect against inadvertent data disclosure.

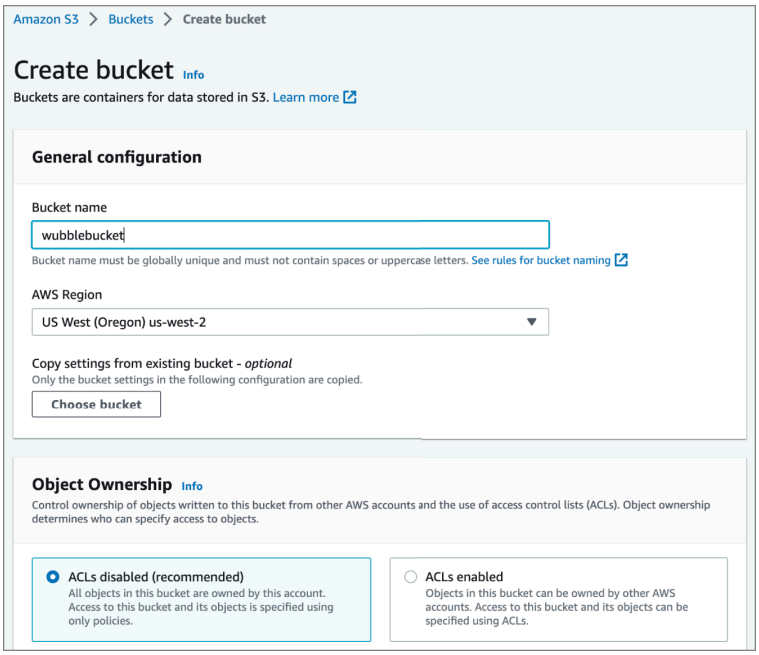

FIGURE 15.18 S3 bucket configuration

Web Application Compromise

Web applications are programmatic interfaces to data, for the most part. It's possible to have a web application where no data is accessible, but it's hard to imagine. Why would you need programmatic access unless you intend to transmit or store data that is going to be manipulated by the server, the client, or both? The problem is there are a lot of ways that web applications can be broken. The Open Worldwide Application Security Project (OWASP) tracks a list of the Top 10 vulnerabilities in web applications. Many of them relate to poor handling of data within the application code. Users can send any data into the application they want, and if the application doesn't handle it correctly, the application may be compromised. This might allow for excess access through the application to the data behind.

Another area where web applications can be compromised is through misconfiguration. An example is enabling public access to the S3 bucket, as shown earlier. Weak network restrictions might be another area of misconfiguration that could allow someone access they shouldn't have. Someone who doesn't understand the controls available within a cloud provider's offerings may not end up using the correct one to implement network access restrictions. While it's true that the controls required for on-premise protections are the same as those for cloud providers, the spe-

cific implementation of those controls may vary from one environment to another.

Misconfiguration is not the only way web applications can be compromised. It can be important to consider the goal of the attacker. They are not always looking for shell access to a system, though it may be fanciful to believe that. In the end, shell access is a means to an end. The end is typically data access. Many attackers today fall into the advanced persistent threat category, and either they are looking for a way to monetize a compromised system or enterprise or they are looking for intellectual property that can be used to compete in the marketplace more successfully. If they can get the data with a good old-fashioned Structured Query Language (SQL) injection attack, that's what they are going to do. The least effort for the most result is usually going to be the approach taken.

There are several recommendations that should be followed when it comes to protecting against web application compromise. One is to ensure you are starting with security in the design phase. Another is to use components that have been assessed for vulnerabilities, ensuring they are kept up-to-date with the latest fixes at all times. Web applications should also be rigorously tested by a third party, even if that third party is someone inside the company with knowledge about security testing. In this case, third party just means someone outside of the development or even quality assurance teams. Additionally, web applications should be protected. This can be done through the use of a web application firewall.

CLOUD PROTECTION

Using any of the cloud service providers, create a virtual machine running any operating system. Only allow network access to ports 80 and 443 using the provided virtual private cloud and/or firewall capabilities.

Credential Compromise

The traditional brute-force attempts against accounts are largely in the past. If there are brute-force attempts happening, it's offline. Credential dumps with hashed passwords can be cracked offline without alerting a business that the attacker is trying to break in. Also, credentials are commonly available in many places online, including through the so-called darknet, which is just an overlay on the Internet using The Onion Router (Tor) to gain access to the systems where this information is available. Once you have a set of credentials, you can try them all. This may not necessarily alert someone you are going after them since a lot of businesses may not be looking for successive failed attempts against different user accounts. They are often looking for multiple failures on the same user account. Credential stuffing, as this technique is called, isn't going to use the same username over and over. The password dumps are more likely to include a single instance of a username, but a lot of different usernames. One password dump from 2019 had more than two billion username and password combinations.

Another way to get username and password combinations is to simply ask. Phishing attacks are common. This is primarily because they are successful, in spite of the large number of phishing messages that get caught up in the Junk net with most mail providers, including the one shown in Figure 15.19. If email messages look convincing enough, users can be tricked into visiting sites, which may also look very convincing. Especially during a busy workday, users may not be the most discerning and can be encouraged to give up their credentials. They are so used to being inundated with one request after another that when one more comes in, they just respond.

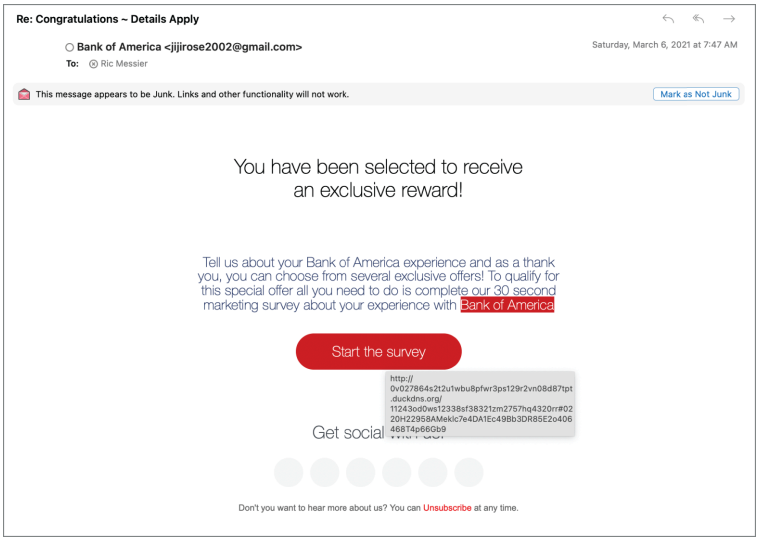

FIGURE 15.19 Phishing message

One common way to protect against phishing messages is to have users go through regular security awareness training. Adopting a partnership approach with the user rather than an adversarial approach is also useful. Detecting phishing messages is hard. If the email protections you have in place aren't enough to catch all the messages, you shouldn't blame the users if they also aren't able to detect malicious messages. Attackers will put a lot of time into making the message look very legitimate.

Other ways to protect against these phishing attacks where the goal may be to gather credentials, some of which can be used to gain access to cloud-based services, are to implement email protections. There are several threat intelligence–based solutions that can be employed to detect even unknown attacks, redirecting malicious messages to a location other than the user's inbox.

While it's not perfect and relies heavily on good implementation, multifactor authentication (MFA) is an essential mitigation against credential-based attacks. MFA can be misused if text messages are used as a second factor or if push messages are used. A push message is when an authentication app sends a request to the user through a notification on a mobile device. The user can confirm the authentication request by just approving it, without any other effort on the part of the user. Push messages can be misused when the attacker continues to try to authenticate, sending the requests to the user until the user gets tired of seeing the request and

just approves it. A better approach is to require the use of a one-time password (OTP) generated from an authentication app or device. Additionally, hardware devices can also be used as a way to implement MFA.

Insider Threat

Interestingly, a lot of businesses see this as their biggest challenge. This is in spite of years of evidence that responding to and recovering from attacks from external entities has cost organizations many billions of dollars over even the last handful of years. However, insider threat is a legitimate problem and more of a problem in some organizations than others. An insider threat is an employee who either steals or damages a business resource. Commonly, this is thought to be a malicious attack, though insiders are probably far more likely to make mistakes that result in damage or loss than they are to engage in a malicious action.

Any employee who has access to resources in a cloud provider environment can cause damage or loss. You should always have controls in place to protect against damage. As always, limiting access to any user to only that needed for the user to perform their job is a good practice. Additionally, logging and monitoring access and actions can help detect damage and loss. This does not mean you have to mistrust all your employees. It means that sometimes bad things happen. Sometimes these things are mistakes. Sometimes they are meant to be harmful or destructive. Being able to observe actions will help identify the actions in progress so you can respond to them and take appropriate action.

TESTING AGAINST THE CLOUD

Testing against a cloud environment is not the same as testing on premise. When you are testing against an on-premise environment, the owner of the network can give you permission. Network blocks would typically all belong to the company you are working with. This means you would typically be able to test against all contiguous addresses. With a cloud provider, that's not true. Additionally, systems and networks belong at least in part to the cloud provider. Some cloud providers have policies for performing penetration testing or other types of security testing. Always make sure you are aware of where the systems under test are located and that you have appropriate permissions to do the types of testing expected of you.

Internet of Things

For decades, computing devices have continued to get smaller. We started with room-sized devices and then went to computers you could comfortably put on your desk, then on your lap, then in the palm of your hand. Today, you may well have computing devices scattered around your house, sometimes in places you aren't aware. As an example, Figure 15.20 shows a light switch that has a small computing device built into it, allowing me to control the switch using an application or my voice, with the

right controller installed on my home network. These are not general-purpose computing devices that would allow you to run any program you want on them. Typically, they are limited-capability devices without a lot of memory or even computing power. After all, they aren't generally being asked to do a lot. The light switch is being asked to either allow power through the switch or prevent power from passing through the switch. The broad collection of all of these devices is called the Internet of Things (IoT).

Today, the smart home has driven the use of a lot of these things that have some computing intelligence and network connections. You may have light switches, appliances, thermostats, healthcare devices, or other types of devices that require some intelligence and networking capability. You may have devices that simply transmit information out of your home network, as may be the case with a sensor. You may also have a device that controls something in your house, like the light switch. As these devices have network capabilities, it means you can interface with them. They are also detectable on your network and, sometimes, to the world at large.

IOT DEVICES

You may run across different definitions of IoT devices. In general, an IoT device is going to be of limited capability, often lacking traditional input/output capabilities (sometimes having neither). Light switches, appliances, and the like definitely qualify for IoT status. A smartphone isn't an IoT device because it is a general-purpose computing platform that can run a variety of programs and also has input/output capabilities.

First, there are a couple of websites that are available to search for IoT devices. You could use Censys.io, which allows you to search for devices in its database based on a long list of characteristics. Another one is Shodan, at `shodan.io`. Shodan maintains a database of IoT devices, allowing those with an account to search for these devices. Figure 15.21 shows some of the quick searches you can perform from Shodan. Selecting one of these searches gives you details about the devices. For instance, looking for Cams gives a list of headers from web requests made to these devices. The response in each case is a set of HTTP headers, telling you the date and time of the request as well as the server information, telling you the software being used to respond to the web requests.

FIGURE 15.20 Internet-enabled light switch

Typically, you wouldn't spend a lot of time on these sites, since finding all of the web-cams in the world, or at least the world according to Shodan, won't be a task you would perform as an ethical hacker. You would be more apt to identify IoT devices through something like a *network scan*. Using `nmap`, we can turn up all the devices on a network, including IoT devices. One example, shown next, is how that light switch shown earlier appears on the network. You can see `nmap` isn't sure what to make of the operating system. According to `nmap`, this is a smartphone of some sort. However, the real clue isn't what `nmap` believes. It's in the media access control (MAC) address. According to the organizationally unique identifier (OUI), the network interface was manufactured by a company called iDevices.

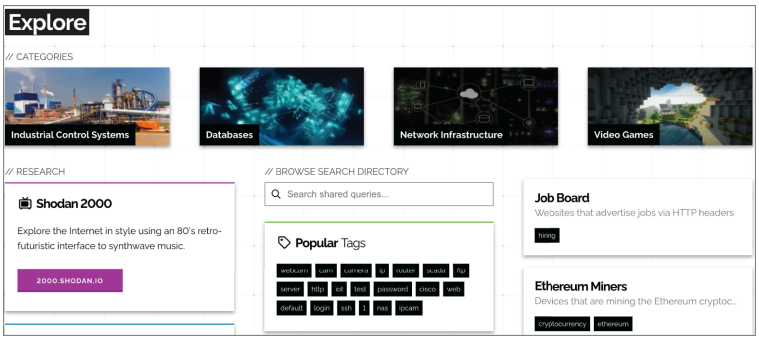

FIGURE 15.21 Shodan website

Nmap Results

```
Nmap scan report for 192.168.4.25
Host is up (0.026s latency).
Not shown: 999 closed ports
PORT   STATE SERVICE
80/tcp open  http
MAC Address: D4:81:CA:5B:8E:CA (iDevices)
Device type: phone|VoIP phone|media device|general purpose
Running (JUST GUESSING): Sony Ericsson Symbian OS 9.X (86%), Palmmicro embedded (85%), Ace
OS CPE: cpe:/h:sonyericsson:p1i cpe:/o:sonyericsson:symbian_os:9.1 cpe:/o:microsoft:windov
Aggressive OS guesses: Sony Ericsson P1i mobile phone (Symbian OS 9.1) (86%), Palmmicro AR
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
```

The company iDevices doesn't make smartphones. It makes smart home devices. One of these (and the only type of iDevices device on this network) is a smart light switch. Where tools are uncertain, as in this case, sometimes you have to resort to another way to obtain the information. What we can see, in addition to the type of device this is, by way of the manufacturer, is there is a port that's open. There appears to be a web server running in this device, which is common for embedded devices (small, single-purpose computing devices where the “program” running is embedded into the firmware). We can do a little probing. The following example shows a connection to this device over port 80 using `netcat` to manage the connection:

```
nc 192.168.4.25 80
GET / HTTP/1.1
Host: 192.168.4.25

HTTP/1.1 404 Not Found
Content-Length: 0
```

Unfortunately, there's no joy here. While there is a web server, it appears to have no content. This may mean there is an API that can be used to control the switch. This may require that you go after endpoints the device knows about using another technique, like the one using Burp Suite shown earlier. There are many other tools available to do testing against APIs. One of these is Postman, which can be used to generate requests to web servers. You can send different types of requests to the web server and include parameters. This will allow you to see how the web server re-

sponds to different requests and parameters. Figure 15.22 shows the use of Postman to send a POST request to the web server embedded in the light switch.

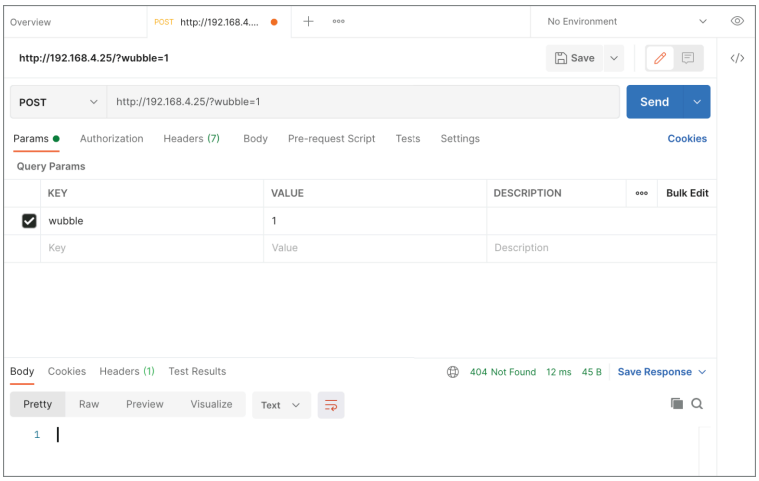

FIGURE 15.22 Postman sending POST request

Many IoT devices will communicate with cloud-based controllers. Some of the cloud providers, like Microsoft Azure, as shown in Figure 15.23, provide services to create IoT-based applications. You can see the different services available in the Azure portal. You could create a central place for any IoT device you may be developing to communicate to. This is not to say that all IoT devices communicate with cloud providers, but with the resources available from cloud providers today, it's a lot easier for companies to make use of existing technology and capabilities. Other cloud providers will have other services that may cater to IoT developers.

FIGURE 15.23 IoT services with Microsoft Azure

All of this is to say that IoT devices are controlled by services and systems out on the Internet somewhere. One way to tackle this is to sniff communications between the IoT device and the controller. This will allow you to determine who the device is communicating with. You may not be able to see the communication stream, however, if it's encrypted.

Of course, this only addresses this circumstance where there is a web server listening but no immediate entry point to the web server. Other

IoT devices may have different ports listening, which may provide other opportunities. The following example shows `nmap` output with open ports as well as a guess at the operating system. This device, based on the manufacturer identification again, is a wireless access point. This does not have a web server listening. Instead, there are two other ports that are open, though the port identification is likely incorrect, since it's based on names for commonly used ports and not necessarily the application that's listening. We can get a little more detail, though, if we use a version scan from `nmap` in addition to just a port scan. You can see that `nmap` has identified the port that it has as a Nessus port using TLS, and we can see some of the certificate details used for encryption. As it's using encryption, we could use a tool like `openssl` to communicate with the port, if we knew more about how to communicate with that port.

Nmap Results

```
Nmap scan report for 192.168.5.14
Host is up (0.0081s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE     VERSION
3001/tcp  open  ssl/nessus?
| ssl-cert: Subject: commonName=eero-GA4642265HE17039/organizationName=eero
| Subject Alternative Name: IP Address:FE80:0:0:0:FABC:EFF:FE05:D1D2
| Not valid before: 2020-06-23T19:46:19
|_Not valid after:  2021-06-23T19:48:19
10001/tcp open  tcpwrapped
MAC Address: F8:BC:0E:05:D1:D2 (eero)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 1 hop
```

Another interesting fact to note is this device is running Linux. Linux can be notoriously difficult to get a version number for since custom kernels will behave differently. If this were using a standard kernel from a Linux distribution, we would know the kernel version in use. However, because this would be a custom kernel for this device and also one that is on an embedded system, it's much harder to identify the actual kernel version. However, based on the fact that this is Linux, it may be possible to identify remote kernel vulnerabilities for possible exploitation.

In the end, when it comes to IoT devices, you can think of them as what they are: computing devices. They aren't general-purpose computing devices in the sense that there is a disk and common userland utilities that you can run any program you want on, but they are computing devices that follow the same rules as other computing devices. There is code that is being executed by a processor, and there is memory. Testing IoT devices may take more time, but you can approach them in a similar way to testing other computing devices.

Fog Computing

Fog computing is a concept that relates to cloud computing and having intelligent edge devices, like those in the Internet of Things. As noted earlier, cloud providers have IoT-based services. This is to allow the edge devices to have controllers in the cloud, allowing collation of data and some processing. Some of these devices, though, may generate a large amount of data. This may not be an ideal circumstance, since it may not be beneficial either from a performance perspective or from a security perspective, to be sending a lot of data to a cloud service from an on-premise device.

Fog computing separates the idea of services from the endpoint device that is used in cloud computing and moves the services closer to the endpoint. This reduces potential latency and also keeps processing closer to where the devices are. Fog computing can be used to support IoT devices, letting these edge devices communicate with controllers that are much closer to them than a fully cloud-based controller would be.

Fog computing has both a data plane and a control plane. The control plane determines how to handle incoming requests, making decisions about what should be done with them. The data plane handles the data, processing it, storing it, aggregating it, or potentially discarding it. This is a design architecture that is also commonly used in network devices like routers. You may think about fog computing as containing a router, with a set of rules (routing tables) for how to move data around, and also the capability to move or handle the data.

Operational Technology

While this has been building for a few years and has been a concern for a long time, you may have started to see more news articles related to industrial control systems, where attackers are targeting things like gas or electrical utility companies. These industrial control systems are referred to as *operational technology*. An industrial control system is a collection of devices, controllers, and interfaces that together allow business operations to supply gas, electricity, water, or some other resource to consumers. This may also include manufacturing systems where devices perform tasks such as, for instance, pasta extrusion to make dried macaroni that can be boxed and sold in grocery stores. All of these types of systems are examples of industrial control systems (ICSs).

One example of a type of industrial control system is supervisory control and data acquisition (SCADA). A SCADA system may include a number of devices that need to be monitored. These devices would be monitored by microcontrollers, which may be programmable logic controllers (PLCs). Each microcontroller would have some supervisory computer that read data off the PLC. The entire SCADA system is multiple tiers, as you can see in Figure 15.24, though this is a subset of the complete Purdue model, covered later. At the top of the network of devices would be a computer that a person would sit at. This is commonly called a *human interface device* (HID). This may also be called a *human–machine interface* (HMI).

Each PLC may have a series of registers, which can be read from or written to. Typically, there would be a protocol that allows a controller to interact with the PLC. One example of this is Modbus. This is a protocol that was originally developed in the days when all interfaces to these devices that may be essentially integrated circuits were handled over serial interfaces. This required that each PLC have a wire connecting it to its controller. The data interface was done over that wire. Today, though, everything is networked. This means Modbus can work just as effectively over a TCP/IP network interface as it can over a traditional serial interface.

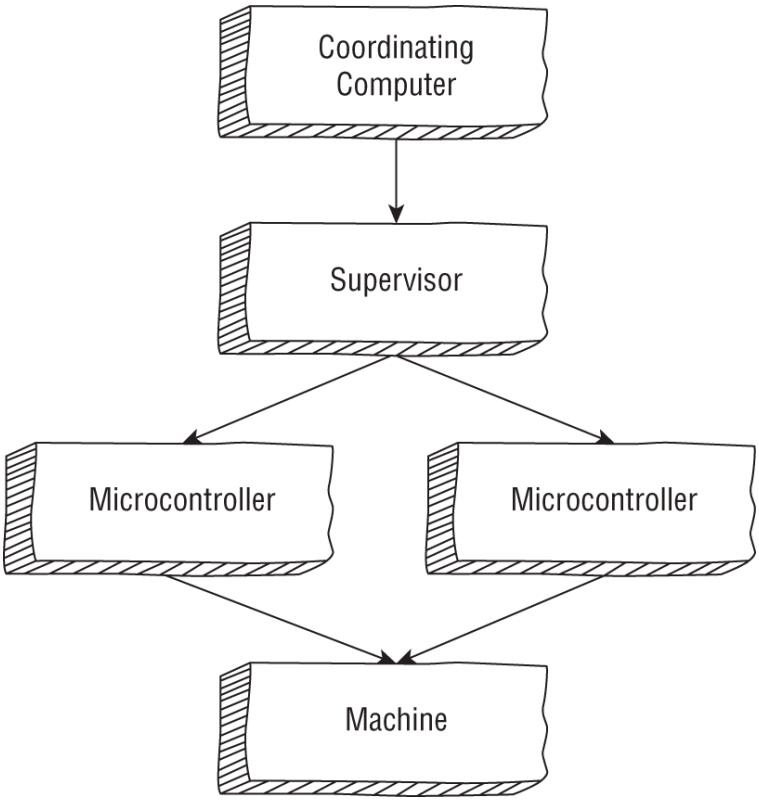

FIGURE 15.24 ICS design

The biggest issue with ICS/SCADA and operational technology is the devices may be fragile because they were designed to perform a specific function, so they can be underpowered and may not be programmed to handle anomalies well. There are also issues around properly implementing common security controls like strong authentication and access control in ICS/SCADA systems. One common problem with these systems is the use of default usernames and passwords. When operational technology was first implemented in many cases, the devices were not networked at all. This meant you needed physical access to all devices in an ICS/SCADA system to perform any sort of attack. This is no longer the case, as it is considered efficient to have, for instance, the HMI device on a network that users can get to remotely, so they don't have to be bound to a computer that sits in an industrial environment.

Where ICS/SCADA systems have problems, or at least can have problems, falls a lot to the weak access control in place. This can allow attackers who can reach these devices from inside the business network to run scripts that can not only read from the PLCs to understand what is happening but also write to the PLCs to cause unexpected behavior within

the industrial control system. In the case of utilities, this could cause significant disruption of customer service.

The Purdue Model

The Purdue model, also known as the Purdue Enterprise Reference Architecture, is an architectural design to guide the design of networks in businesses using computer-integrated manufacturing (CIM), allowing the business to fully automate their operations. This is a design that has been used for industrial control systems as a way of separating functions to keep operational technology (OT) away from information technology (IT). OT devices are the ones mentioned previously that may include programmable logic controllers and the devices that monitor and manage them. As these can be potentially fragile devices, it is useful to have a way of designing networks that contain them to keep the fragile devices as protected as possible, while still allowing operators to maintain the devices in a cost-effective way.

Figure 15.25 shows the reference architecture and the different layers, or zones. Starting at the bottom is level 0, or the physical process zone. This is where sensors and actuators are—the devices that do the work of manufacturing. Above that is level 1 where the intelligent devices that interface with the sensors and actuators are. This is the programmable logic controllers (PLCs) or remote terminal units (RTUs). Above that is the control systems zone, where you would have the devices that communicate with the PLCs and RTUs. This would be the supervisory control and data acquisition (SCADA) systems, as well as the human–machine interface (HMI) devices. The final zone of the operational technology layers is the site operations/control layer. This is where manufacturing operations management (MOM) systems would be, or manufacturing execution systems (MES). These would interface with the HMIs and SCADA systems in the lower zone.

Above that zone are the IT-based layers. Between the IT and OT should be some segmentation. Some people recommend a demilitarized zone (DMZ) to protect IT systems that may need to interface with OT systems. In an ideal world, all the OT systems would be completely air-gapped so no one could get to them unless they were in the physical location where those systems were. That's not very practical in today's world, however. The IT layers include all the infrastructure that may be necessary for the OT systems to operate effectively, including centralized identity and access management (IAM), as well as database servers for storing essential data related to operations that may be needed for dashboards or other reporting.

Today's operational technology may often be IoT devices, which makes it much more difficult to completely segment OT in the way described by the Purdue model. However, while the different layers may not be implemented in the way suggested by the Purdue model, the segmentation of resources into distinct zones to protect sensitive or fragile systems or devices is still an essential concept in implementing OT-based infrastructure.

FIGURE 15.25 Purdue Enterprise Reference Architecture

Summary

Cloud computing is the latest iteration of a long-established method of organizations handling computing needs: it's outsourcing. Cloud computing vendors, often called cloud providers, provide different levels of services, each with different levels of responsibility for the customer. There is infrastructure as a service, which puts most of the responsibility in the customer's hands. There is also platform as a service, which shares the responsibility between the provider and the customer relatively equally. Finally, there is storage as a service and software as a service where the provider takes care of most everything except access control and complete data protection. If you drop a sensitive file into a public bucket, there is nothing the cloud provider can do for you.

Cloud providers offer a lot of capabilities that most organizations wouldn't be able to do by themselves. In some cases, you could implement a public cloud to get on-demand services like the ones offered by cloud providers, but when it comes to developing hosted applications, there are some features that are harder to implement on your own. The cloud provider has put a lot of engineering resources into making these services as easily consumable as possible. This opens the door to something called cloud-native design. With cloud-native design, there is a large emphasis on virtualization and abstraction from systems. You get away from just standing up a virtual machine with a complete operating system and en-

vironment and move more toward developing your application using virtual functions and sometimes virtualized applications, called containers.

Cloud-native design also tends to make use of service-oriented architectures, sometimes called microservices. Traditional application design used a monolithic design where everything was in a single application. The microservice model is more modular, with a lot of small services (application components) sending messages to one another over a data bus, implemented in software. This modularity and all the virtualization means you can implement applications using a responsive design, sometimes called elastic design. Using this approach, you can stand up instances of your services on demand to handle load and then turn them off when they are not needed, saving costs because computing providers charge for use.

Just because you are moving to a cloud provider doesn't mean you have to stop doing all the protection and detection that you were doing when the services were on your premises. You still have to do all the blocking and tackling, meaning implementation of common and reasonable security controls. The threats are the same in a cloud environment as they are on-premise, though the pathways for the threats to manifest may be different. Common cloud threats include poorly implemented access management, data breach, web application compromise, credential compromise, and insider threat.

The Internet of Things (IoT) is a collection of small, commonly single-purpose devices. These devices don't have the traditional input/output capabilities of general-purpose computing devices. They may not have the same elements you think of a computer having, at least in a traditional sense. They likely don't have storage in the way you think about it where there is storage that allows data to be arbitrarily written to and read from. Programs may reside in hardware rather than be modifiable on a storage device. None of this is to say, however, that these devices can't be compromised. In fact, history has shown they can be and have been. If they are networked, and they can't be an IoT device if they are not networked, you can send network messages to them, and there may be applications listening for network connections. These can be the starting points, just as these are the starting points for other system compromise.

Operational technology (OT) is a collection of systems and devices that are used to control industrial systems. This may be technology used to control utilities, but it may also be systems that are used for any sort of process automation, including robotic devices used for manufacturing. As these devices are connected to broader networks, it exposes them to attack by adversaries around the world. The Purdue model is a way of describing network design into segments to protect sensitive or fragile OT system or devices. The Purdue model helps to keep capabilities separate and is also conceptual from the perspective of keeping different devices and functions logically distinct.

Review Questions

You can find the answers in the appendix.

1. Which of these is not an example of an IoT device?
   A. Chromebook
   B. iDevices light switch
   C. Nest thermostat
   D. Amazon Echo
2. Why is REST a common approach to web application design?
   A. HTML is stateless.
   B. HTTP is stateless.
   C. HTML is stateful.
   D. HTTP is stateful.
3. Which of these cloud offerings relies on the customer having the most responsibility?
   A. Software as a service
   B. Platform as a service
   C. Storage as a service
   D. Infrastructure as a service
4. Which of these is less likely to be a common element of cloud-native design?
   A. Microservice architecture
   B. Automation
   C. Virtual machines
   D. Containers
5. Which of these is not an advantage of using automation in a cloud environment?
   A. Consistency
   B. Fault tolerance
   C. Repeatability
   D. Testability
6. What common element of a general-purpose computing platform does an IoT not typically have?
   A. Memory
   B. Processor
   C. External keyboards
   D. Programs
7. If you wanted to share documents with someone using a cloud provider, which service would you be most likely to use?
   A. Software as a service
   B. Platform as a service
   C. Storage as a service
   D. Infrastructure as a service
8. What tool could you use to identify IoT devices on a network?
   A. Cloudscan
   B. `nmap`
   C. Samba
   D. Postman
9. What might you be most likely to use to develop a web application that used a mobile application for the user interface?
   A. NoSQL database

B. Data bus
C. Microservices
D. RESTful API

10. Which of these might be a concern with moving services to a cloud provider, away from on-premise services?
    A. Multiple accounts per user
    B. Lack of transport layer encryption
    C. Inability to implement security controls
    D. Lack of access to necessary operating systems and hardware
11. What modern capability does fog computing support?
    A. Cloud-native design
    B. IoT
    C. Access management
    D. Grid computing
12. Which of these is an example of grid computing?
    A. SETI@home
    B. OWASP Top 10
    C. Shodan.io
    D. Thingful.net
13. What is a botnet an example of?
    A. Cloud-native design
    B. RESTful processing
    C. Grid computing
    D. Fog computing
14. Which of these would be least likely as a vulnerability to serverless web applications?
    A. Insecure third-party components
    B. Misconfiguration
    C. Access control
    D. Command injection
15. No matter what cloud-based service model is being used, which of these is the customer always responsible for?
    A. Patching
    B. Operating system installation
    C. Application management
    D. Data
16. What devices would you find at the lowest layer of the Purdue model and, as a result, should be the best protected?
    A. Sensors and actuators
    B. Programmable logic controllers
    C. Remote terminal units
    D. Human–computer interfaces
17. What aspect of the Purdue model is considered essential from a security perspective?
    A. Human–computer interfaces
    B. Segmentation
    C. IT and OT interfaces
    D. Layered model matches OSI
18. What is a good mitigation against credential compromise in cloud-based services?
    A. Long passwords
    B. Using only software as a service

C. Requiring MFA
D. Using serverless functions

19. Which of these might be a common problem with a cloud-based service leading to data exposure?
A. Public buckets
B. Insider threat
C. Selecting the wrong provider
D. Only using software as a service

20. Which of these would not be used to automate deployment of systems into a cloud environment?
A. Templates
B. Ansible
C. PowerShell
D. Containers